

# Aero: A Pluggable Congestion Control for QUIC

Jashanjot Singh Sidhu  
Concordia University, Canada  
jashanjot.sidhu@mail.concordia.ca

Abdelhak Bentaleb  
Concordia University, Canada  
abdelhak.bentaleb@concordia.ca

## ABSTRACT

QUIC is rapidly emerging as the de-facto standard for HTTP Adaptive Streaming (HAS). QUIC relies on heuristic-based congestion control algorithms which were predominantly designed for TCP and thus have poor generalizability, ultimately degrading the user's Quality of Experience (QoE). Existing learning-based solutions for TCP are not pluggable and require a lot of engineering work, impacting their integration with QUIC. To tackle these challenges, we develop Aero—the first learning-based plug-and-play congestion control algorithm for QUIC that considers the varying network statistics and can easily be integrated with any QUIC implementation. To analyze the performance of Aero, we conduct a series of comprehensive trace-driven experiments and evaluate its efficiency not only from a congestion control perspective but also the impact it has on the client-driven adaptive bitrate scheme (ABR). Experimental results demonstrate that Aero improves VMAF by ~12% with ~65% less rebuffering for low latency live streaming sessions compared to its competitors. Moreover, Aero excels in terms of delivery rate and delay across different network conditions.

## CCS CONCEPTS

• Information systems → Multimedia streaming.

## KEYWORDS

CC, QUIC, ABR, DRL, LLL, QoE.

## ACM Reference Format:

Jashanjot Singh Sidhu and Abdelhak Bentaleb. 2025. Aero: A Pluggable Congestion Control for QUIC. In *The 35th edition of the Workshop on Network and Operating System Support for Digital Audio and Video (NOSSDAV '25)*, March 31–April 4, 2025, Stellenbosch, South Africa. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3712678.3721878>

## 1 INTRODUCTION

The most recent Global Internet Phenomena Report [1] underscores the dominance of video streaming services on the Internet, contributing to 65% of the total traffic during 2023. This surge in video traffic is attributed to major technology companies such as YouTube, Netflix, and Meta. These industry giants predominantly utilize HTTP Adaptive Streaming (HAS) [4] to deliver their content to an extensive audience worldwide. HAS allows video streaming

clients to adjust the video quality based on network conditions, offering users a smoother and uninterrupted streaming experience.

For years, Transmission Control Protocol (TCP) has been the widely adopted transport protocol for HAS-based traffic. Recently, the emergence of Google's QUIC [26] transport protocol marks a pivotal shift in network communication and media delivery [14]. For instance, YouTube and Meta are increasingly serving video traffic over QUIC. QUIC offers several advanced features such as multiplexing, 0-RTT, prioritization, and seamless connection migration. These features make QUIC a compelling alternative to TCP, particularly for HAS-based streaming media. QUIC leverages UDP as its underlying protocol and is designed to support HTTP/3. This combination allows QUIC to provide a more efficient/secure transport mechanism for video streaming services, offering an improved viewer Quality of Experience (QoE) in comparison to conventional TCP-based solutions [24, 26, 36].

Typically, a server-side (*a.k.a* sender) Congestion Control (CC) Algorithm regulates the sending rate based on network capacity, while an Adaptive Bitrate (ABR) Scheme on the client-side (*a.k.a* receiver) selects the ideal bitrate and resolution pair for upcoming segments based on one or multiple factors such as available bandwidth and buffer level. Although many widely adopted QUIC implementations [13] exist like Quiche, Quinn and Aioquic, all of them use heuristic CC algorithms that were primarily designed and handcrafted for TCP. Some of the most widely employed heuristic-based CC algorithms are Cubic [15], NewReno [17] and BBR2 [8]. These conventional algorithms fell short in adapting to the diverse network scenarios [28, 29], paving the way for the emergence of learning-based CC for TCP. By incorporating deep reinforcement learning (DRL) techniques into the CC logic of TCP, existing implementations [2, 43] have shown a considerable improvement for TCP. Despite their promising performance enhancements, these DRL-based CC algorithms require kernel patches. This not only demands substantial engineering efforts for integration with QUIC but also undermines the essence of CC in QUIC as a user mode [21].

### 1.1 Problem and Motivation

To demonstrate the limitations of heuristic-based CC algorithms in QUIC, we conducted some experiments using *Vegvisir* framework [18], which consists of three Docker containers: server-side (Quiche), traffic shaper (tc Netem), and client-side (Quic-go [33] with goDash player [35] for HAS streaming), connected via Docker-compose networks to regulate traffic flow. We used Pensieve ABR [30] and the LTE Belgium [40] trace (avg. bandwidth: 3.3 Mbps) to vary client-server bandwidth. The experiment was run 3 times, with each run lasting 120s, and the average of the results was taken. As illustrated in Figure 1, BBR2, Cubic, and Reno demonstrate a failure to increase their delivery rate [9], with their congestion windows (cwnd) remaining far below optimal levels. Although BBR2 adapts in the initial 10-15 seconds, it subsequently fails to adapt as the link capacity increases. Our proposed solution, Aero, on the other

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
NOSSDAV '25, March 31–April 4, 2025, Stellenbosch, South Africa  
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 979-8-4007-1469-6/25/03  
<https://doi.org/10.1145/3712678.3721878>

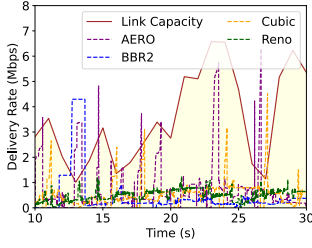


Figure 1: Delivery rate vs. link capacity.

hand, is able to dynamically adjust to the available link capacity and has a higher average delivery rate, indicating its efficiency and versatility to varying network conditions.

To address these challenges, we propose Aero, a DRL-based CC solution that leverages network statistics (*i.e.*, throughput, delay, jitter, and packet loss) to dynamically adjust cwnd, enhancing adaptability to link capacity and aiding ABR schemes in optimizing bitrate decisions for improved QoE. To our knowledge, Aero is the first pluggable learning-based CC for QUIC and this paper is the first study to evaluate the impact a QUIC CC holds on ABR. This paper presents four key contributions:

- (1) We propose Aero, a server-side ABR-agnostic learning-based pluggable CC for QUIC, that dynamically adjusts the cwnd based on current network conditions and assists ABR schemes in their bitrate decisions for better QoE.
- (2) We model the cwnd adjustment problem in QUIC as a Markov Decision Process (MDP) and design an A2C neural network (NN) architecture with PPO and GAE to effectively learn new policies, providing quick adaptation.
- (3) We implement Aero in Quiche and conduct extensive experiments that include various ABR schemes, network traces, CC algorithms, and video content, for low latency live streaming mode. Experimental results demonstrate that Aero outperforms its competitors in terms of QoE and its metrics, delivery rate, and delay.
- (4) We plug Aero into Aioquic and Quinn to evaluate its fairness and competitiveness. Experimental results demonstrate that Aero is QUIC-friendly among multi-clients.

The rest of the paper is organized as follows. Section 2 highlights some existing CC solutions. Section 3 describes Aero CC, followed by a performance evaluation in Section 4. Finally, Section 5 concludes the paper.

## 2 RELATED WORK

**Server-driven heuristic-based CC.** The heuristic-based CC algorithms are categorized into loss-based, delay-based, and hybrid. Cubic, NewReno, and Tahoe [22] are some of the most popular loss-based algorithms. They update the cwnd primarily based on packet loss alone and thus fail to adapt to variations in delay and throughput. Vegas [5], Copa [3], and FAST [41] are the widely adopted delay-based algorithms. They update the cwnd primarily based on the delay in the network and thus cannot adapt well to packet loss and link capacity. The most popular hybrid algorithms are BBR [7] and BBR2 [8], which take both packet loss and delay into account but as we saw in Section 1.1, they fail to adapt to the link capacity.

**Server-driven learning-based CC.** PBQ [46] (no available code)

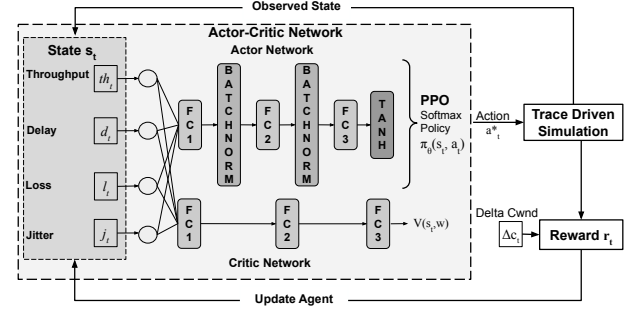


Figure 2: Aero DRL architecture.

is the only known learning-based CC mechanism built on top of BBR for QUIC. It offers two levels of control but still requires a heuristic-based CC algorithm as the first level of control. For TCP, there are few learning-based CC algorithms. Remy [42] tries to dynamically adjust the sending rate based on network feedback. Similarly, Aurora [23] develops a simple fully connected NN for CC tasks. Orca [2] utilizes A2C network with RL policies combined with Cubic. Similarly, Sage [43] is an offline off-policy DRL-based technique that requires no interaction with the environment. The main issue with these TCP learning-based approaches is that they require kernel patches at the transport layer, thereby making their integration with QUIC in user mode quite difficult.

## 3 AERO SOLUTION

In contrast to the conventional CC algorithms that use Additive Increase Multiplicative Decrease (AIMD) principle for updating the cwnd, during a congestion event, Aero adopts a DRL approach to dynamically adjust the cwnd, ensuring adaptability to diverse network conditions. In particular, Aero utilizes an actor-critic (A2C) approach with PPO for the policy selection and update. PPO's clipped objective function ensures stable training by limiting policy updates, allowing the network to learn more effectively and adapt to diverse environments. The overall A2C NN architecture of Aero is shown in Figure 2.

### 3.1 Problem Formulation

At each time step  $t$ , Aero decides the optimal congestion window (denoted by  $\text{cwnd}_t^*$ ) for the next state (denoted by  $s_{t+1}$ ) based on the current state (denoted by  $s_t$ ) by selecting a multiplicative factor (denoted by  $A^\alpha$ ). The cwnd update problem is mathematically described as follows:

$$\begin{cases} \text{find } \text{cwnd}_t^*(\pi), \forall t \in \{1, \dots, k\} \\ \arg \max_{\pi} r_t(\pi) \\ \text{s.t. } A \in [1.2, 3] \text{ C.1 and } \alpha \in [-1, 1] \text{ C.2,} \end{cases} \quad (1)$$

where  $\text{cwnd}_t^*$  is the optimal congestion window based on the current network statistics for  $s_{t+1}$ .  $\pi$  is an RL policy that helps to decide the best congestion window update by updating both  $A$  and  $\alpha$ ,  $k$  is the total number of steps, and  $r$  is the step reward (defined in (2)). The problem (1) is described as a single-agent decision problem aimed at finding the best congestion window update with respect to C1 and C2, wherein the agent has access to states from the environment at each time step  $t$ . We fine-tuned the multiplicative factor  $A$  for cwnd update, finding that setting it above 3 led to overly drastic updates, making recovery impossible. Setting it below 1.2

places it in the same league as AIMD-based updates.  $\alpha$  here is bounded by the *tanh* activation function itself. This setup aligns with the formulation of an MDP [12]. The MDP process is a 4-tuple  $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$ , where each element is defined as:

- $\mathcal{S} = \{s_1, \dots, s_k\}$  is a set of finite and discrete agent states.
- $\mathcal{A} = \{a_1, \dots, a_k\}$  is a set of finite and continuous actions.
- $\mathcal{P} = \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$  is the state transition probability function  $\mathcal{P}(s_{t+1}|s_t, a_t)$  from the state  $s_t$  to state  $s_{t+1}$  when action  $a_t$  is taken.
- $\mathcal{R} = \{r_1, \dots, r_k\}$  is the set of expected immediate rewards, which depends on the state and action taken by the agent at every time step  $t$ .  $\mathcal{R}(s_t, a_t, s_{t+1}) (= r_t)$  is the step reward.

At each time step  $t \in \{1, \dots, k\}$ , the agent has access to the observations from the environment in the form of a state vector  $\vec{s}_t$ . It selects the right multiplicative factor for cwnd that maximizes the accumulated discounted advantage (denoted by  $\Lambda_t$  and defined in Section 3.2). The Aero model solves the MDP problem using a single agent A2C NN with clipped PPO, generalized advantage function (GAE), and RMSProp optimizer for policy ( $\pi^*$ ).

### 3.2 Aero DRL Offline Training

We utilize the dataset provided by the Microsoft RL Bandwidth Estimation Challenge [32] to train the A2C NN Aero model. This dataset consists of a series of peer-to-peer Microsoft Teams audio/video calls. From Aero’s perspective, the list of metrics selected is summarized as follows: (1) *Receiving Rate* (bps); (2) *Delay* (ms); (3) *Packet Jitter* (ms); (4) *Packet Loss Ratio*; (5) *Packets Probability*; (6) *Bottleneck Capacity* (bps); all from [32]. To enhance generalization across network conditions, we introduced linear noise to delay  $[-2, 100]$  ms and loss  $[0, 5]\%$ , augmenting the agent’s observation space. The dataset was based on discrete audio/video calls and thus, resulted in a sub-optimal discrete state-action space with PPO. By adjusting the reward based on bottleneck capacity (Algorithm 1), we transformed it into a semi-continuous space. The RL agent interacts with an environment defined by state space  $\mathcal{S}$  and action space  $\mathcal{A}$ . At each time step  $t$ , the agent observes state  $s_t \in \mathcal{S}$ , takes action  $a_t \in \mathcal{A}$ , and receives reward  $r_t \in \mathcal{R}$ . The goal is to find the optimal policy  $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$  that maximizes the reward.

► **Input State.** At each time step  $t$ , the agent takes as input the current state defined by  $s_t = \{th_t, d_t, l_t, j_t\}$ , comprised of throughput (bps), delay (ms), packet loss ratio (%) and jitter (ms). The  $s_t$  is fed to the A2C as a 4-dimensional vector  $\vec{s}_t$  as shown in Figure 2. To avoid the potential bias in decision-making that can result from directly feeding state values into the NN network, we employ *min-max* normalization to scale the state vector.

► **Action Space.** The action space  $\mathcal{A}$  is defined as the predicted cwnd for the next state  $s_{t+1}$ . For each time step  $t$ , the Aero policy  $\pi^*$  maps the  $\vec{s}_t$  to a continuous action space  $\mathcal{A}$  and selects  $a_t \in \mathcal{A}$ . ► **Output.** The Aero actor model returns a single scalar ( $\alpha$ ) to be used as a multiplier for the next cwnd. The policy  $\pi^* : s_t \rightarrow a_t^*$  maps the  $s_t$  to the best  $a_t^*$  based on the output of the actor network. The critic network outputs a single scalar as well, indicating the value function  $V^\pi(s_t)$  for the current state  $s_t$ . The cwnd for  $s_{t+1}$  can be computed as:  $cwnd_{t+1} = A^\alpha \times cwnd_t$ , where  $A$  is the base multiplier and  $\alpha$  is the power factor for  $A$ .  $A$  is bounded from [1.2, 3] and is a learned variable for the Aero model. Algorithm 1 describes

---

#### Algorithm 1 Aero reward and cwnd base factor update.

---

```

1: procedure UPDATE( $c\hat{w}nd_t, \mathbb{E}[cwnd_t], cwnd_{max}, r_t, \phi, \rho, A$ )
  ►  $c\hat{w}nd_t$  is predicted cwnd for current  $s_t$  from  $s_{t-1}$ 
  ►  $\mathbb{E}[cwnd_t]$  is expected cwnd for current  $s_t$ 
  ►  $cwnd_{max}$  is the maximum possible cwnd for current  $s_t$ 
  ►  $r_t$  is the current reward for time step  $t$ 
  ►  $\phi$  is the cwnd difference threshold for  $\Delta cwnd$ 
  ►  $\rho$  is the extra gain/penalty factor (empirically set to 0.1)
  ►  $A$  is the base factor for cwnd update given by  $A^\alpha$ 
2: Initialize  $r_t = 0$  and estimate  $\mathbb{E}[cwnd_t]$  from the bottleneck capacity
3: Compute  $\Delta cwnd = |c\hat{w}nd_t - \mathbb{E}[cwnd_t]|$ 
4:   if  $c\hat{w}nd_t > cwnd_{max}$  then
5:      $r_t = r_t - 2 \times \rho, A = A - \rho$ 
6:   end if
7:   if  $c\hat{w}nd_t == \mathbb{E}[cwnd_t]$  then
8:      $r_t = r_t + 1.5 \times \rho$ 
9:   else if  $c\hat{w}nd_t > \mathbb{E}[cwnd_t]$  then
10:    if  $\Delta cwnd < \phi$  then
11:       $r_t = r_t - \rho, A = A - \rho/2$ 
12:    else
13:       $r_t = r_t - 2 \times \rho, A = A - \rho$ 
14:    end if
15:  else
16:    if  $\Delta cwnd < \phi$  then
17:       $r_t = r_t + \rho, A = A + \rho/2$ 
18:    else
19:       $r_t = r_t + 2 \times \rho, A = A + \rho$ 
20:    end if
21:  end if
22: end procedure

```

---

how Aero updates the reward  $r$  and therefore the base factor  $A$  for cwnd. In the end, we take the average value of  $A$  which is collected from all experience trajectories accumulated during Aero training. ► **NN Architecture.** The Aero NN architecture (Figure 2) consists of two networks: the actor and the critic. Each network uses three fully connected (FC) layers, with the actor network being augmented by two batch normalization layers for heightened training stability. Each FC layer contains 256 neurons and utilizes Leaky Rectified Linear Unit (Leaky ReLU()) activation function. The output of the last FC layer in the actor network uses *tanh* activation function, providing an output  $\in [-1, 1]$ . In the critic network  $\vec{s}_t$  and  $a_t$  are concatenated in the first FC layer for each time step  $t$ . The output is then derived from the last FC layer.

► **Reward Function.** At each time step  $t$ , the agent takes action  $a_t$ , and a corresponding reward  $r_t$  is computed to evaluate  $a_t$  in the current state  $s_t$ , guiding the agent toward the optimal policy  $\pi^*$ . For Aero, we define a reward function (2) that linearly combines throughput ( $th_t$ ), delay ( $d_t$ ), packet loss ratio ( $l_t$ ), and jitter ( $j_t$ ):

$$r_t = r_t + th_t - w_1 \times d_t - w_2 \times l_t - w_3 \times j_t, \quad (2)$$

Initially,  $r_t$  is adjusted as per Algorithm 1, and the final  $r_t$  is calculated based on  $s_t$ . To determine  $w_1, w_2, w_3$ , and  $w_4$ , 10% of the training dataset was used for tuning. Results show that  $w_3 = 2$  in all cases. For a high video packet ratio,  $w_1 = 4, w_2 = 5$ ; for a high audio packet ratio,  $w_2 = 4$ ; and for a high probing packet ratio,  $w_2 = 2$  and  $w_1 = 3$ . This highlights the greater impact of packet loss on videos than on probing packets.

► **Policy Gradient and Training Algorithm.** The goal of Aero is to refine policies by increasing the probabilities of high-reward samples from trajectories while reducing those of failure samples from sub-optimal paths. At time step  $t$ , the agent maps  $\vec{s}_t$  to the optimal

multiplicative factor  $\alpha_t$  to compute the cwnd for the next state  $s_{t+1}$  using the improved policy  $\pi: \pi_\theta^*(\alpha_t, a_t) \rightarrow [-1, 1]$ , maximizing the advantage function:

$$\Lambda_t = \sum_{i=t}^{T_{\pi_\theta}} r_i + \gamma \delta_t + \gamma \lambda \Lambda_{t+1}, \quad (3)$$

$$a_t = \arg \max_a \mathbb{E}[\Lambda_t(\alpha_t, a)],$$

where  $T_{\pi_\theta}$  is the batch size,  $\gamma \in [0, 1]$  is the discount factor,  $\delta_t$  is the TD error ( $V_{s_{t+1}} - V_{s_t}$ ), and  $\lambda$  is the GAE parameter.  $\theta$  is the policy parameter and  $\pi_\theta(\alpha_t, a_t)$  is the probability for action  $a_t$  taken in state  $s_t$ . TD error provides immediate feedback, while GAE incorporates short-term and long-term effects for efficient policy updates.

PPO uses mini-batches to compute a clipped surrogate objective for stable policy updates:

$$L^{clip}(\theta) = -\mathbb{E}_t \left[ \min \left( \mathcal{R}_t(\theta) \hat{\Lambda}_t, \text{clip}(\mathcal{R}_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{\Lambda}_t \right) \right], \quad (4)$$

where  $\mathcal{R}_t(\theta)$  is the probability ratio between new and old policies,  $\hat{\Lambda}_t$  is the advantage, and  $\epsilon$  limits updates.

The value function loss is:

$$L^{value}(\theta) = \frac{1}{2} \mathbb{E}_t \left[ (V(s_t; \theta) - G_t)^2 \right], \quad (5)$$

where  $G_t$  is the observed return. Entropy loss encourages exploration:

$$L^{ent}(\theta) = \mathbb{E}_t \left[ \sum_a \pi(a|s_t; \theta) \log \pi(a|s_t; \theta) \right], \quad (6)$$

The total gradient is:

$$\nabla_\theta L^{total}(\theta) = \nabla_\theta L^{clip}(\theta) + \nabla_\theta L^{value}(\theta) + \nabla_\theta L^{ent}(\theta), \quad (7)$$

and the parameter update follows:

$$\theta \leftarrow \theta - \omega \nabla_\theta L^{total}(\theta), \quad (8)$$

where  $\omega$  is the learning rate. This process is repeated per epoch, resulting in the optimal policy  $\pi^*$ .

## 4 PERFORMANCE EVALUATION.

### 4.1 Aero Implementation

**4.1.1 Aero Parameter Selection.** Aero was trained on a audio/video calls based dataset [32], as outlined in Section 3.2. Given that the platforms for training and testing were distinct, all traces were utilized for training purposes. Training parameters are crucial for optimizing any NN architecture. Therefore, the hyperparameter tuning for the Aero model was conducted using Bayesian optimization techniques. The optimal set of parameters for Aero, as observed during this process, are summarized in Table 1.

**4.1.2 Offline Training.** Aero's NN architecture was implemented in Pytorch 2.1.1+cu121 with Python 3.10.12. The training was done in a continuous trace driven fashion as highlighted in Section 3.2.

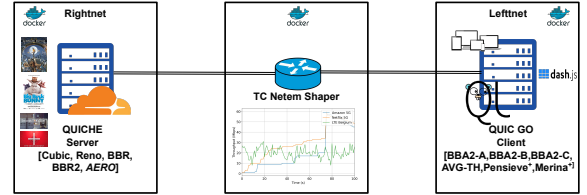
**4.1.3 Online Testing.** During Aero testing, the average base multiplier  $A$  from training was 1.87, with cwnd bounds set based on the min-max bandwidths in the HAS MPD. We evaluated Aero using the *Vegvisir* framework [18] for QUIC trace-driven simulations, employing a Quiche-based server and a Quic-go client, as shown in Figure 3. The Quic-go client, integrated with a headless goDASH player [35], facilitated efficient metric extraction. Network emulation leveraged tc-Netem [16] with real-world traces.

**Table 1: Aero training/testing parameters.**

|     | Parameter                            | Symbol                 | Value                                |
|-----|--------------------------------------|------------------------|--------------------------------------|
| A2C | Discount Factor                      | $\gamma$               | 0.95                                 |
|     | Advantage estimator                  | $\lambda$              | 0.9                                  |
|     | Actor & Critic learning rates        | $\omega, \bar{\omega}$ | $3 \times 10^{-4}, 3 \times 10^{-3}$ |
| PPO | Clip parameter                       | $\epsilon$             | 0.2                                  |
|     | Entropy Coefficient                  | $\delta$               | 2.6                                  |
|     | PPO policy update interval, steps    | $T_\theta, n_s$        | 725, 6                               |
| NN  | Batch size                           | $B_s$                  | 128                                  |
|     | Exploration size, number of episodes | $\theta, n_e$          | 29, 25                               |
|     | Number of epochs                     | itr                    | 38000                                |

**Table 2: Experimental details.**

| Video | FPS | Bitrate Range (Mbps) | Seg. Dur. (s) | Max buffer (s) |
|-------|-----|----------------------|---------------|----------------|
| NR    | 60  | 0.5 - 40             | 4             | 12             |



**Figure 3: Vegvisir framework.**

### 4.2 Methodology

**4.2.1 Video Sample and Parameters.** 3DMark Night Raid (NR) [39] video was used during Aero testing for low latency live (LLL) streaming mode, encoded using 10 bitrate levels ranging between 288p@0.5Mbps–2160p@40Mbps (4K), the details of which can be found in Table 2. To ensure accuracy, all trace-driven simulations were conducted three times, with each run lasting  $\sim 120$  seconds. The results presented are the average of these runs.

**4.2.2 Network Traces.** We used three network traces, 5G Amazon [34], 5G Netflix [34] and LTE Belgium [40] with average bandwidths (and sampling interval) of 24.9 Mbps (1s), 33.3 Mbps (1s) and 20 Mbps (1s), respectively. Note that the 5G traces are sub-6GHz's and not mmWave.

**4.2.3 CC Algorithms and Metrics.** Heuristic-based implementations in Quiche were used for comparison. Cubic and Reno served as baselines for loss-based algorithms, while BBR2 was chosen for hybrid algorithms. Most learning-based CC algorithms are implemented in TCP and require kernel patches, which complicates their integration and undermines the advantages of user-mode CC in QUIC. Consequently, these algorithms were excluded from our baseline comparisons. To evaluate Aero, we compared it against the selected CC algorithms using two key metrics: mean normalized delivery rate and delay. Values for delivery rate and delay were extracted from qlog [31] files and linearly interpolated over time.

**4.2.4 ABR Algorithms and Metrics.** We used two heuristic-based and two learning-based ABR schemes. The heuristic-based schemes include BBA2-XLDouble [19] (BBA2-C), a buffer-based approach with an auxiliary stall prevention logic to reduce playback stalls, and AVG-TH [19], a throughput-based scheme using the average of the most recent 20 throughput samples for ABR decisions. For learning-based ABR, we used Pensieve [30] and Merina [25], which were originally designed for 6 bitrate levels. We re-trained both for 10 bitrate levels and more epochs to align with our video samples. Additionally, we incorporated BBA2-C's stall prevention logic into Pensieve (Pensieve+) and Merina (Merina+).

To have comprehensive comparison amongst combination of all algorithms (CC+ABR), we used three well-known QoE models: Duanmu [11], Yin [44], and Yu [45]. These QoE models take two or multiple metrics as input such as selected bitrate, rebuffering number (nReb), rebuffering ratio (RD), number of segments downloaded in ultra-HD quality (nSeg4K), startup delay, bitrate switches with different objectives. The output of each model is a QoE value that ranges between 0 (bad) and 1 (good) after normalization for the remaining models. We also used VMAF [27] and PSNR [38] as perceptual quality metrics. The  $QoE^{itu}$  metric [20] was omitted due to its unreliability, as it does not support resolutions higher than 1080p and the extended ITU-P.1204 implementation for 4K content is in its initial stages, cannot parse frame by frame, and is only suitable for short videos with 10 seconds maximum [6, 37].

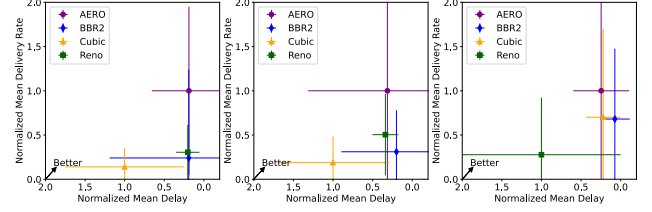
**4.2.5 Experimental Setup.** Our setup consisted of a physical machine running Ubuntu 22.04.4 LTS, equipped with an NVIDIA GeForce RTX 3060, a 24-core CPU, and 64 GB memory. During testing, the client, shaper, and server were deployed in Docker containers, forming a cohesive streaming system. These containers were connected via two Docker-compose networks, with traffic routed through the shaper to regulate flow, as illustrated in Figure 3.

### 4.3 Results and Analysis

**4.3.1 Aero CC for QUIC Results.** In this section, we compare the Aero CC efficiency against heuristic-based CC algorithms. Figure 4 shows the normalized mean delivery rate versus normalized mean delay for Aero, BBR2, Cubic, and Reno, in different network traces, and for LLL streaming mode. In the 5G Amazon trace (left), Aero, BBR2, and Reno have comparable delays with Cubic having the highest delay. However, in terms of delivery rate, Aero performs significantly better than all other CC. Similar results are observed for the 5G Netflix trace (center) as well as the LTE Belgium trace (right) with Reno having the highest delay instead of Cubic for LTE Belgium trace. The limited buffer capacity in LLL amplifies the impact of latency variations on BBR2’s CC decisions, potentially leading to a sub-optimal delivery rate. Cubic and Reno are purely loss-based CC algorithms that aggressively increase the delivery rate until a loss occurs. The frequent oscillations between high and low delivery rates following packet losses contribute to their sub-optimal performance and inconsistency across different network conditions. In contrast, Aero demonstrated superior performance across all traces. Overall, Aero attained  $\sim 161\%$  higher average delivery rate compared to BBR2 while only having a slightly higher ( $\sim 18\%$ ) delay. Cubic and Reno had the highest average delay. As further validated by our findings in Section 4.3.2, delay and delivery rate are equally important factors for better QoE in LLL mode.

These results indicate that the existing heuristic-based CC algorithms lack adaptability to varying network conditions, which impacts the QoE negatively, as shown in Table 3. In contrast, Aero performs consistently across all network traces, highlighting its generalizability and superior performance.

**4.3.2 Impact of Aero on ABR over QUIC Results.** In this section, we compare the Aero CC impact on heuristic-based as well as learning-based ABR schemes. Table 3 shows the comparison in terms of different QoE metrics for different network traces and for LLL streaming mode.



**Figure 4: Normalized delivery rate vs. delay in LLL for network traces 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right), respectively.**

Across all traces, Aero+Pensieve<sup>+</sup> has the highest average bitrate (col.3) while having minimal rebuffering (col.4). In 5G Amazon trace, Aero+Merina<sup>+</sup> has the highest VMAF (col.1), which is  $\sim 135\%$  higher than its counterpart. This substantial improvement is especially significant for streaming 4K content. Although the frequency of rebuffering events is lower for BBR2+Merina<sup>+</sup> compared to its counterpart, the duration of these stalls is notably higher, as illustrated in Figure 5. Aero+Pensieve<sup>+</sup> (runner-up) also shows significant improvement in VMAF scores *w.r.t.* its counterpart. Aero+BBA2-C performs comparably to BBR2+BBA2-C on all fronts. Cubic, Reno, and BBR2, when combined with BBA2-C, and similarly BBR2+AVG-TH, show comparable performance. However, all of these fall significantly short in terms of VMAF.

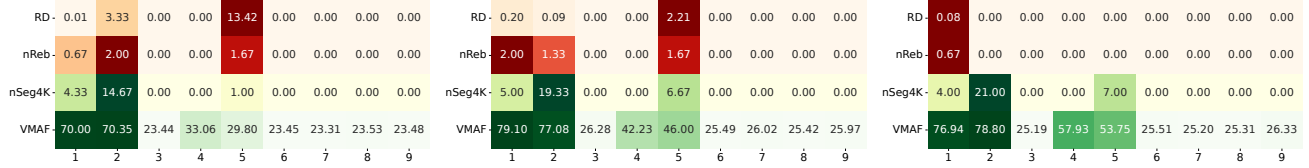
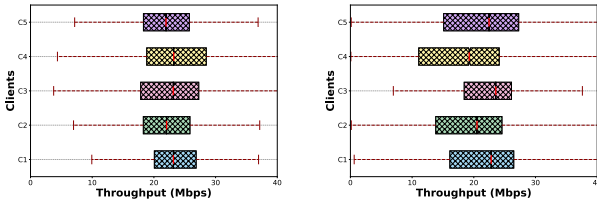
Similar results are observed for the 5G Netflix trace, where Aero+Pensieve<sup>+</sup> achieves the highest VMAF score. BBR2+Pensieve<sup>+</sup> falls significantly behind in VMAF compared to its counterpart. The findings for the LTE Belgium trace align with those of the 5G traces. Note that Figure 5 also highlights the number of segments downloaded in 4K resolution (nSeg4K) across all network traces for the given combinations. Aero+Merina<sup>+</sup> and Aero+Pensieve<sup>+</sup> downloaded significantly more high quality segments for the same duration, which can be attributed to the fact that they provide substantially higher average bitrates. BBR2, in combination with the same ABRs, downloads fewer 4K segments as it lags behind in terms of average bitrate.

Overall, Aero+Pensieve<sup>+</sup> and Aero+Merina<sup>+</sup> achieves the highest VMAF scores with minimal rebuffering. Their counterparts significantly lag behind in terms of VMAF by  $\sim 70\%$ . In addition, they also select the highest average bitrate along with the highest PSNR (col.2) scores, thereby providing a smooth streaming experience. On all metrics, Aero+BBA2-C performs comparably *w.r.t.* to its counterparts BBR2, Cubic, and Reno with the same ABR as baseline. BBR2+Merina<sup>+</sup> provides the highest rebuffering duration, negatively impacting the QoE. These results are consistent with our findings that Aero was able to maintain a much higher delivery rate with minimal delay, which in turn assists ABR’s decision making ability and emphasizes on Aero’s adaptability across all network traces and ABR algorithms, making it truly ABR-agnostic.

**4.3.3 QUIC Friendliness and Pluggability of Aero.** We extended our architecture from Figure 3 to support 5 clients, where a single server is connected to all clients with a one-to-many mapping, with all clients simultaneously requesting the video content. This modification allows us to evaluate the QUIC friendliness of Aero and to observe how multiple QUIC flows compete with each other. While we initially aimed to extend Quiche for this purpose,

**Table 3: Average results of the QoE and its metrics for different network traces. ↑: higher is better (green), ↓: lower is better (green), lowest performance (red), and mode: LLL.**

|             |                            | Avg. Perceptual Quality (VMAF) | Avg. Perceptual Quality (PSNR) | Avg. Selected Bitrate (Mbps) | Average RD (%) | Average Duanmu | Average Yu | Average Yin |
|-------------|----------------------------|--------------------------------|--------------------------------|------------------------------|----------------|----------------|------------|-------------|
| 5G Amazon   | Aero+Pensieve <sup>+</sup> | 70.00                          | 38.91                          | 3.94 ↑                       | 0.01           | 1.00 ↑         | 1.00 ↑     | 1.00 ↑      |
|             | Aero+Merina <sup>+</sup>   | 70.35 ↑                        | 39.94 ↑                        | 2.90                         | 3.33           | 0.24           | 0.61       | 0.16        |
|             | Aero+BBA2-C                | 23.44                          | 32.00                          | 0.54                         | 0.00 ↓         | 0.59           | 0.16       | 0.27        |
|             | BBR2+Pensieve <sup>+</sup> | 33.06                          | 33.06                          | 0.74                         | 0.00 ↓         | 0.62           | 0.18       | 0.12        |
|             | BBR2+Merina <sup>+</sup>   | 29.80                          | 32.77                          | 0.55                         | 13.42          | 0.19           | 0.17       | 0.14        |
|             | BBR2+BBA2-C                | 23.45                          | 31.79                          | 0.52                         | 0.00 ↓         | 0.59           | 0.16       | 0.26        |
|             | Cubic+BBA2-C               | 23.31                          | 31.93                          | 0.52                         | 0.00 ↓         | 0.59           | 0.16       | 0.28        |
|             | Reno+BBA2-C                | 23.53                          | 31.92                          | 0.50                         | 0.00 ↓         | 0.58           | 0.15       | 0.28        |
|             | BBR2+AVG-TH                | 23.48                          | 31.91                          | 0.63                         | 0.00 ↓         | 0.61           | 0.18       | 0.27        |
| 5G Netflix  | Aero+Pensieve <sup>+</sup> | 79.10 ↑                        | 40.64                          | 6.65 ↑                       | 0.20           | 1.00 ↑         | 1.00 ↑     | 0.32        |
|             | Aero+Merina <sup>+</sup>   | 77.08                          | 41.43 ↑                        | 4.40                         | 0.09           | 0.89           | 0.58       | 0.00        |
|             | Aero+BBA2-C                | 26.28                          | 32.37                          | 1.62                         | 0.00 ↓         | 0.61           | 0.30       | 0.78        |
|             | BBR2+Pensieve <sup>+</sup> | 42.23                          | 34.21                          | 1.24                         | 0.00 ↓         | 0.59           | 0.20       | 0.21        |
|             | BBR2+Merina <sup>+</sup>   | 46.00                          | 35.63                          | 1.10                         | 2.21           | 0.26           | 0.17       | 0.06        |
|             | BBR2+BBA2-C                | 25.49                          | 32.09                          | 1.09                         | 0.00 ↓         | 0.56           | 0.20       | 0.55        |
|             | Cubic+BBA2-C               | 26.02                          | 32.27                          | 1.09                         | 0.00 ↓         | 0.57           | 0.19       | 0.44        |
|             | Reno+BBA2-C                | 25.42                          | 32.13                          | 1.02                         | 0.00 ↓         | 0.56           | 0.17       | 0.31        |
|             | BBR2+AVG-TH                | 25.97                          | 32.43                          | 1.23                         | 0.00 ↓         | 0.57           | 0.26       | 1.00 ↑      |
| LTE Belgium | Aero+Pensieve <sup>+</sup> | 76.94                          | 39.89                          | 5.68 ↑                       | 0.08           | 1.00 ↑         | 1.00 ↑     | 0.00        |
|             | Aero+Merina <sup>+</sup>   | 78.80 ↑                        | 41.40 ↑                        | 3.24                         | 0.00 ↓         | 0.82           | 0.54       | 0.00        |
|             | Aero+BBA2-C                | 25.19                          | 32.28                          | 1.38                         | 0.00 ↓         | 0.59           | 0.39       | 1.00 ↑      |
|             | BBR2+Pensieve <sup>+</sup> | 57.93                          | 36.47                          | 2.76                         | 0.00 ↓         | 0.77           | 0.58       | 0.00        |
|             | BBR2+Merina <sup>+</sup>   | 53.75                          | 36.56                          | 1.00                         | 0.00 ↓         | 0.56           | 0.24       | 0.29        |
|             | BBR2+BBA2-C                | 25.51                          | 32.13                          | 1.43                         | 0.00 ↓         | 0.60           | 0.40       | 0.92        |
|             | Cubic+BBA2-C               | 25.20                          | 32.27                          | 1.39                         | 0.00 ↓         | 0.59           | 0.39       | 0.99        |
|             | Reno+BBA2-C                | 25.31                          | 32.20                          | 1.41                         | 0.00 ↓         | 0.59           | 0.39       | 0.96        |
|             | BBR2+AVG-TH                | 26.33                          | 32.17                          | 2.56                         | 0.00 ↓         | 0.77           | 0.57       | 0.46        |

**Figure 5: VMAF vs. nReb, nSeg4K, RD analysis in LLL for network traces 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right). 1: Aero+Pensieve<sup>+</sup>, 2: Aero+Merina<sup>+</sup>, 3: Aero+BBA2-C, 4: BBR2+Pensieve<sup>+</sup>, 5: BBR2+Merina<sup>+</sup>, 6: BBR2+BBA2-C, 7: Cubic+BBA2-C, 8: Reno+BBA2-C, 9: BBR2+AVG-TH.****Figure 6: QUIC friendliness of Aioquic (left) and Quinn (right) for AERO with 5 clients over 5G Amazon trace.**

its lack of support for a multi-client setup due to the absence of an asynchronous or multi-threaded API for data transmission proved a limitation [10]. However, since the metrics required for the Aero architecture are readily accessible across various QUIC implementations (usually in the congestion controller class), integrating Aero into different QUIC stacks is straightforward. As a result, we successfully incorporated Aero into other QUIC implementations, specifically Quinn and Aioquic, highlighting the pluggability and ease of integration of Aero. Although PBQ-enhanced QUIC [46] is also a DRL-based architecture, the code/GitHub for that is not publicly available. Still, PBQ is built on top of BBR, and many QUIC implementations like Aioquic and Kwik do not support it by default.

Figure 6 illustrates competing QUIC flows for Aero serving multiple clients for Aioquic and Quinn. Aero provided nearly identical mean throughput to each client for both Aioquic and Quinn. In addition, we also computed the Jain fairness index and its value was found to be 1 and 0.97 for Aioquic and Quinn, respectively. This underscores that Aero is QUIC-friendly.

## 5 CONCLUSIONS

This paper presented Aero, the first pluggable learning-based congestion control for QUIC. Aero employs a DRL-based approach to dynamically adjust its congestion window, enabling swift adaptation to prevailing network conditions. Experimental findings illustrate that Aero exhibits superior performance in accommodating diverse network conditions, as evidenced by its enhanced delivery rate and comparable delay. It also helps in delivering a better user experience by assisting the ABR in making informed decisions by adjusting its sending rate for LLL mode. Moreover, Aero seamlessly integrates with any QUIC implementation, offering QUIC-friendly features that ensure fair bandwidth sharing among multiple clients.

**Acknowledgments.** This research is funded by the Natural Sciences & Engineering Research Council (NSERC)-Discovery Grant RGPIN-2023-04744 and NSERC Alliance (ALLRP 586434-23)-Mitacs Accelerate (IT36792) grant.

## REFERENCES

- [1] 2023. THE GLOBAL INTERNET PHENOMENA REPORT JANUARY 2023. <https://www.sandvine.com/phenomena>.
- [2] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. 2020. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *ACM SIGCOMM*.
- [3] Venkat Arun and Hari Balakrishnan. 2018. Copa: Practical {Delay-Based} congestion control for the internet. In *USENIX NSDI*.
- [4] Abdelhak Bentaleb, Bayan Taani, Ali C Begen, Christian Timmerer, and Roger Zimmermann. 2018. A survey on bitrate adaptation schemes for streaming media over HTTP. *IEEE Communications Surveys & Tutorials* 21, 1 (2018), 562–585.
- [5] Lawrence S Brakno, Sean W O'Malley, and Larry L Peterson. 1994. TCP Vegas: New techniques for congestion detection and avoidance. In *ACM SIGCOMM*.
- [6] c3m3gyanesh. 2023. Working with Longer duration video. [https://github.com/Telecommunication-Telemedia-Assessment/bitstream\\_mode3\\_p1204\\_3/issues/31](https://github.com/Telecommunication-Telemedia-Assessment/bitstream_mode3_p1204_3/issues/31).
- [7] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. 2017. BBR: Congestion-based congestion control. *Commun. ACM* 60, 2 (2017), 58–66.
- [8] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. 2019. TCP BBR v2 Alpha/Preview Release.
- [9] Sheng Chen, Y. Li, and X. Zhang. 2024. *Delivery Rate Estimation for Congestion Control*. Technical Report draft-cheng-iccr-delivery-rate-estimation-01. IETF. <https://www.ietf.org/archive/id/draft-cheng-iccr-delivery-rate-estimation-01.html> Work in Progress.
- [10] Inc. Cloudflare. 2025. Quiche-server can't handle more than one connection #1554. <https://github.com/cloudflare/quiche/issues/1554>.
- [11] Zhengfang Duanmu, Abdul Rehman, and Zhou Wang. 2018. A quality-of-experience database for adaptive video streaming. *IEEE Transactions on Broadcasting* 64, 2 (2018), 474–487.
- [12] Frédéric Garcia and Emmanuel Rachelson. 2013. Markov decision processes. *Markov Decision Processes in Artificial Intelligence* (2013), 1–38.
- [13] QUIC Working Group. 2024. *QUIC Implementations*. <https://github.com/quicwg/base-drafts/wiki/Implementations> Accessed: 2024-07-21.
- [14] Zafer Gurel, Tugce Erkilic Civelek, Atakan Bodur, Senem Bilgin, Deniz Yenicieri, and Ali C Begen. 2023. Media over QUIC: Initial testing, findings and results. In *ACM MM Sys*.
- [15] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS operating systems review* 42, 5 (2008), 64–74.
- [16] Stephen Hemminger et al. 2005. Network emulation with NetEm. In *Linux conf au*, Vol. 5. 2005.
- [17] Tom Henderson, Sally Floyd, Andrei Gurtov, and Yoshifumi Nishida. 2012. *The NewReno modification to TCP's fast recovery algorithm*. Technical Report.
- [18] Joris Herbots, Mike Vandersanden, Peter Quax, and Wim Lamotte. 2023. Vegvisir: A testing framework for HTTP/3 media streaming. In *ACM MMSys*.
- [19] Joris Herbots, Arno Verstraete, Maarten Wijnants, Peter Quax, and Wim Lamotte. 2023. Cross that boundary: Investigating the feasibility of cross-layer information sharing for enhancing ABR decision logic over QUIC. In *Proceedings of the 33rd Workshop on Network and Operating System Support for Digital Audio and Video*. 50–57.
- [20] ITU-T. 2024. ITU-T P.1203: Subjective video quality assessment methods for multimedia applications. <https://github.com/itu-p1203/itu-p1203>.
- [21] J. Iyengar and I. Swett. 2021. *RFC 9002: QUIC Loss Detection and Congestion Control and Stream*. RFC 9002. IETF. <https://www.rfc-editor.org/rfc/rfc9002.html>
- [22] Sabbar Insaif Jasim. 2018. Old TAHOE and TAHOE Congestion Control Algorithms for TCP. *Network* 1, d1 (2018), d1.
- [23] Nathan Jay, Noga Rotman, Brighton Godfrey, Michael Schapira, and Aviv Tamar. 2019. A deep reinforcement learning perspective on internet congestion control. In *International Conference on Machine Learning*. PMLR, 3050–3059.
- [24] Arash Molavi Kakhki, Samuel Jero, David Choffnes, Cristina Nita-Rotaru, and Alan Mislove. 2017. Taking a long look at QUIC: an approach for rigorous evaluation of rapidly evolving transport protocols. In *proceedings of the 2017 internet measurement conference*. 290–303.
- [25] Nuowen Kan, Yuankun Jiang, Chenglin Li, Wenrui Dai, Junni Zou, and Hongkai Xiong. 2022. Improving generalization for neural adaptive video streaming via meta reinforcement learning. In *ACM MM*.
- [26] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasnic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, et al. 2017. The quic transport protocol: Design and internet-scale deployment. In *ACM SIGCOMM*.
- [27] Zhi Li, Christos Bampis, Julie Novak, Anne Aaron, Kyle Swanson, Anush Moorthy, and JD Cock. 2018. VMAF: The journey continues. *Netflix Technology Blog* 25, 1 (2018).
- [28] Jinting Lin, Lin Cui, Yuxiang Zhang, Fung Po Tso, and Quanlong Guan. 2019. Extensive evaluation on the performance and behaviour of TCP congestion control protocols under varied network scenarios. *Computer Networks* 163 (2019), 106872.
- [29] Josip Lorincz, Zvonimir Klarin, and Julije Ožegović. 2021. A comprehensive overview of TCP congestion control in 5G networks: Research challenges and future perspectives. *Sensors* 21, 13 (2021), 4510.
- [30] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In *ACM SIGCOMM*.
- [31] Robin Marx, Wim Lamotte, Jonas Reynnders, Kevin Pittevels, and Peter Quax. 2018. Towards QUIC debuggability. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*. 1–7.
- [32] Microsoft. 2023. *Bandwidth Estimation Challenge*. <https://www.microsoft.com/en-us/research/academic-program/bandwidth-estimation-challenge/> Accessed: 2023.
- [33] quic-go. 2024. quic-go: An implementation of the QUIC protocol in Go. <https://github.com/quic-go/quic-go>.
- [34] Dario Raca, Dylan Leahy, Cormac J Sreenan, and Jason J Quinlan. 2020. Beyond throughput, the next generation: A 5G dataset with channel and context metrics. In *ACM MM*.
- [35] Dario Raca, Maëlle Manificier, and Jason J Quinlan. 2020. Godash—go accelerated framework for rapid prototyping. In *IEEE QoMEX*.
- [36] David Schinazi, Fan Yang, and Ian Swett. 2020. Chrome is deploying HTTP/3 and IETF QUIC. Retrieved October 8 (2020), 2020.
- [37] slhck. 2020. Memory issues. [https://github.com/Telecommunication-Telemedia-Assessment/bitstream\\_mode3\\_p1204\\_3/issues/16](https://github.com/Telecommunication-Telemedia-Assessment/bitstream_mode3_p1204_3/issues/16).
- [38] Alexander Tanchenko. 2014. Visual-PSNR measure of image quality. *Journal of Visual Communication and Image Representation* 25, 5 (2014), 874–878.
- [39] Babak Taraghi, Hadi Amirpour, and Christian Timmerer. 2022. Multi-codec ultra high definition 8K MPEG-DASH dataset. In *ACM MM*.
- [40] Jeroen Van Der Hooft, Stefano Petrangeli, Tim Wauters, Rafael Huysegems, Patrice Rondao Alfaced, Tom Bostoen, and Filip De Turck. 2016. HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks. *IEEE Communications Letters* 20, 11 (2016), 2177–2180.
- [41] David X Wei, Cheng Jin, Steven H Low, and Sanjay Hegde. 2006. FAST TCP: motivation, architecture, algorithms, performance. *IEEE/ACM transactions on Networking* 14, 6 (2006), 1246–1259.
- [42] Keith Winstein and Hari Balakrishnan. 2013. Tcp ex machina: Computer-generated congestion control. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 123–134.
- [43] Chen-Yu Yen, Soheil Abbasloo, and H Jonathan Chao. 2023. Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control. In *ACM SIGCOMM*.
- [44] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *ACM SIGCOMM*.
- [45] Li Yu, Tammam Tillo, and Jimin Xiao. 2017. QoE-driven dynamic adaptive video streaming strategy with future information. *IEEE Transactions on Broadcasting* 63, 3 (2017), 523–534.
- [46] Zhifei Zhang, Shuo Li, Yiyang Ge, Ge Xiong, Yu Zhang, and Ke Xiong. 2023. PBQ-Enhanced QUIC: QUIC with Deep Reinforcement Learning Congestion Control Mechanism. *Entropy* 25, 2 (2023), 294.